

3.PolyAlphabetic Cipher

AIM :To implement a **Polyalphabetic Substitution Cipher** in C for encrypting and decrypting a plaintext message using multiple substitution alphabets generated from a repeating key.

Algorithm

Encryption Algorithm

1. Start.
2. Read the plaintext message.
3. Read the encryption key.
4. Convert the key into uppercase.
5. For each alphabetic character in the plaintext:
 - Convert the plaintext character into a numerical value from 0 to 25.
 - Convert the corresponding key character into a numerical value from 0 to 25.
 - Apply the formula:

$$C = (P + K) \% 26$$

6. Convert the resulting value back into a character.
7. Repeat the key when the end of the key is reached.
8. Keep spaces and special characters unchanged.
9. Display the encrypted text.
10. Stop.

Decryption Algorithm

1. Start.
2. Read the encrypted text and key.
3. For each alphabetic character:
 - Convert the ciphertext character and key character into numerical values.
 - Apply the formula:

$$P = (C - K + 26) \% 26$$

4. Convert the result back into a character.
5. Repeat the key when required.
6. Display the decrypted text.
7. Stop.

Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char text[100], key[100];
    int i, j = 0;
    int keyLength;

    printf("Enter the text: ");
    fgets(text, sizeof(text), stdin);

    printf("Enter the key: ");
    scanf("%s", key);

    keyLength = strlen(key);

    // Encryption
    for (i = 0; text[i] != '\0'; i++) {

        if (isalpha(text[i])) {

            if (isupper(text[i])) {
                text[i] = ((text[i] - 'A') +
                    (toupper(key[j % keyLength]) - 'A')) % 26 + 'A';
            }

            else {
                text[i] = ((text[i] - 'a') +
                    (toupper(key[j % keyLength]) - 'A')) % 26 + 'a';
            }
        }
    }
}
```

```

        j++;
    }
}

printf("\nEncrypted Text: %s", text);

// Reset key index for decryption
j = 0;

// Decryption
for (i = 0; text[i] != '\0'; i++) {

    if (isalpha(text[i])) {

        if (isupper(text[i])) {
            text[i] = ((text[i] - 'A') -
                (toupper(key[j % keyLength]) - 'A') + 26) % 26 + 'A';
        }

        else {
            text[i] = ((text[i] - 'a') -
                (toupper(key[j % keyLength]) - 'A') + 26) % 26 + 'a';
        }

        j++;
    }
}

printf("Decrypted Text: %s", text);

return 0;

```

}

Output

The screenshot shows an IDE window titled "D:\CSA5112 - Cryptography\LAB PROGRAMS\Poly Cipher.cpp - [Executing] - Embarcadero Dev-C++ 6.3". The code in the editor implements a Caesar cipher with functions for encryption and decryption. The console output shows the program running successfully, taking user input for text and key, and displaying the encrypted and decrypted results.

```
print("Enter the key: ");
scanf("%s", key);

keylength = strlen(key);

// Encryption
for (i = 0; text[i] != '\0'; i++) {
    if (isalpha(text[i])) {
        if (isupper(text[i])) {
            text[i] = ((text[i] - 'A') +
                (toupper(key) % keylength) - 'A') % 26 + 'A';
        }
        else {
            text[i] = ((text[i] - 'a') +
                (toupper(key) % keylength) - 'A') % 26 + 'a';
        }
    }
    i++;
}

printf("Encrypted Text: %s", text);

// Reset key index for decryption
j = 0;

// Decryption
for (i = 0; text[i] != '\0'; i++) {
    if (isalpha(text[i])) {
        if (isupper(text[i])) {
            text[i] = ((text[i] - 'A') -
                (toupper(key) % keylength) - 'A') % 26 + 26 + 'A';
        }
        else {
            text[i] = ((text[i] - 'a') -
                (toupper(key) % keylength) - 'A') % 26 + 26 + 'a';
        }
    }
    i++;
}

printf("Decrypted Text: %s", text);

return 0;
```

Compiler: Resources: Compile Log: Debug: Find Results: Console: Close

Abort Compilation

Shorten compiler pat

Errors: 0
Warnings: 0
Output Filename: D:\CSA5112 - Cryptography\LAB PROGRAMS\Poly Cipher.exe
Output Size: 324.4697265625 KiB
Compilation Time: 0.23s

Line: 64 Col: 2 Sel: 0 Lines: 64 Length: 1423 Insert Done parsing in 0.079 seconds

```
D:\CSA5112 - Cryptography\U
Enter the text: attack at dawn
Enter the key: lemon

Encrypted Text: Lxfopv ef mnhr
Decrypted Text: attack at dawn

-----
Process exited after 22.06 seconds with return value 0
Press any key to continue . . . |
```